

4.11 스택킹 앙상블(Stacking Ensemble)

목차

1. 스택킹 앙상블 개요

1. 스택킹이란
2. 기반 모델과 메타 모델
3. 다른 앙상블과의 차이

2. 기본 스택킹 모델 구현

1. 데이터 준비와 개별 모델 생성
2. 개별 모델 학습과 성능 측정
3. 예측 결과를 피쳐로 변환
4. 메타 모델 학습과 평가
5. 기본 스택킹의 치명적 문제

3. CV 세트 기반의 스택킹

1. 개선 원리
2. 기반 데이터 생성 함수
3. 4개 모델에 함수 적용
4. 메타 모델용 데이터 결합
5. 최종 메타 모델 학습과 평가

4. 핵심 요약 및 머신러닝 활용 포인트

1. 스택킹 앙상블 개요

1.1 스택킹이란

스택킹(Stacking)은 개별 알고리즘의 예측 결과를 결합한다는 점에서 배깅·부스팅과 비슷하지만, 결정적인 차이가 있습니다.

개별 알고리즘의 예측 결과 데이터 세트를 최종적인 메타 데이터 세트로 만들어, 별도의 ML 알고리즘으로 최종 학습을 수행하고 예측하는 방식입니다. 즉 **예측 결과 자체를 새로운 피처로 삼아** 다시 학습하는 것입니다.

■ 직관적인 비유

여러 전문가(기반 모델)에게 진단을 받은 뒤, 그 진단서들을 모아 **최종 판단을 내리는 종합 전문의(메타 모델)**가 있는 구조입니다. 종합 전문의는 원본 환자 데이터가 아니라 **"전문가 A는 양성, 전문가 B는 음성이라고 했다"**는 정보만 보고 판단합니다. 어떤 전문가가 어떤 상황에서 더 믿을 만한지를 학습하는 셈입니다.

1.2 기반 모델과 메타 모델

구분	역할	입력 데이터
기반 모델 (Base Model)	원본 데이터를 학습하고 예측을 수행하는 개별 모델들	원본 피처 (본 실습에서는 30개)
메타 모델 (Meta Model)	기반 모델들의 예측값을 학습해 최종 예측 을 수행	기반 모델들의 예측 결과 (본 실습에서는 4개)

스택킹을 구현하려면 두 종류의 모델이 필요하며, **많은 개별 모델이 반드시 있어야** 합니다. 스택킹은 일반적으로 **많은 개별 모델을 필요 하므로**, 성능을 조금이라도 더 끌어올려야 하는 **대회 등에서 주로 사용**됩니다.

1.3 다른 앙상블과의 차이

기법	결합 방식	결합 규칙
보팅	예측 레이블/확률	고정된 규칙 (다수결 또는 평균)
배깅	예측 레이블	고정된 규칙 (다수결)
부스팅	순차적 가중치 보정	알고리즘 내부 규칙
스택킹	예측 결과를 피처화	메타 모델이 규칙을 학습

■ 스택킹의 강점

보팅은 **"평균을 낸다"**는 규칙이 고정되어 있습니다. 반면 스택킹은 **"어떤 모델의 예측에 얼마나 비중을 둘지"**를 데이터로부터 학습합니다. 특정 상황에서 어떤 모델이 더 정확한지를 메타 모델이 스스로 파악하므로, 이론적으로 더 유연합니다.

2. 기본 스택킹 모델 구현

2.1 데이터 준비와 개별 모델 생성

```
import numpy as np

from sklearn.neighbors import KNeighborsClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import AdaBoostClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.linear_model import LogisticRegression

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score

cancer_data = load_breast_cancer()

X_data = cancer_data.data
y_label = cancer_data.target

X_train , X_test , y_train , y_test = train_test_split(X_data , y_label , test_size=0.2 , random_state=0)
```

줄	코드	상세 설명
3~6	KNeighborsClassifier / RandomForestClassifier / AdaBoostClassifier / DecisionTreeClassifier	기본 모델로 쓸 4가지 서로 다른 알고리즘 을 임포트합니다. 거리 기반(KNN), 배깅(RF), 부스팅(AdaBoost), 단일 트리(DT)로 성격이 모두 달라 스택킹 효과를 기대할 수 있습니다.
7	from sklearn.linear_model import LogisticRegression	메타 모델 로 사용할 로지스틱 회귀입니다. 메타 모델은 입력 피처가 적고 단순하므로 가벼운 선형 모델 이 적합합니다.
13~16	cancer_data = load_breast_cancer() 등	유방암 데이터를 로딩합니다. 569건 × 30피처의 이진 분류 문제입니다.
18	train_test_split(..., test_size=0.2, random_state=0)	학습 455건 / 테스트 114건 으로 분리합니다. 앞 장들과 달리 random_state=0 입니다.

```
# 개별 ML 모델을 위한 Classifier 생성.
knn_clf = KNeighborsClassifier(n_neighbors=4)
rf_clf = RandomForestClassifier(n_estimators=100, random_state=0)
dt_clf = DecisionTreeClassifier()
ada_clf = AdaBoostClassifier(n_estimators=100)

# 최종 Stacking 모델을 위한 Classifier 생성.
lr_final = LogisticRegression(C=10)
```

줄	코드	상세 설명
2	knn_clf = KNeighborsClassifier(n_neighbors=4)	가장 가까운 4개 이웃 의 다수결로 분류합니다.
3	rf_clf = RandomForestClassifier(n_estimators=100, random_state=0)	결정 트리 100개의 배깅 앙상블 입니다.
4	dt_clf = DecisionTreeClassifier()	단일 결정 트리입니다. random_state 를 지정하지 않아 실행할 때마다 결과가 조금씩 달라질 수 있습니다.

5	<code>ada_clf = AdaBoostClassifier(n_estimators=100)</code>	에이다부스트 입니다. 이전 모델이 틀린 데이터에 가중치를 부여하며 100개의 약한 학습기를 순차 학습합니다.
8	<code>lr_final = LogisticRegression(C=10)</code>	메타 모델입니다. C 는 규제 강도의 역수 로, 값이 클수록 규제가 약해집니다. 기본값 1보다 큰 10을 주어 학습 데이터에 더 적합하도록 설정했습니다.

2.2 개별 모델 학습과 성능 측정

```
# 개별 모델들을 학습.
knn_clf.fit(X_train, y_train)
rf_clf.fit(X_train, y_train)
dt_clf.fit(X_train, y_train)
ada_clf.fit(X_train, y_train)
```

줄	코드	상세 설명
2~5	각 모델의 <code>fit()</code>	4개 모델을 모두 동일한 학습 데이터(455건) 로 학습합니다. 보팅과 같은 구조입니다.

```
# 학습된 개별 모델들이 각자 반환하는 예측 데이터 셋을 생성하고 개별 모델의 정확도 측정.
knn_pred = knn_clf.predict(X_test)
rf_pred = rf_clf.predict(X_test)
dt_pred = dt_clf.predict(X_test)
ada_pred = ada_clf.predict(X_test)

print('KNN 정확도: {0:.4f}'.format(accuracy_score(y_test, knn_pred)))
print('랜덤 포레스트 정확도: {0:.4f}'.format(accuracy_score(y_test, rf_pred)))
print('결정 트리 정확도: {0:.4f}'.format(accuracy_score(y_test, dt_pred)))
print('에이다부스트 정확도: {0:.4f} :'.format(accuracy_score(y_test, ada_pred)))
```

줄	코드	상세 설명
2~5	각 모델의 <code>predict(X_test)</code>	테스트 데이터 114건 에 대한 예측 레이블을 각각 연습니다. 각 결과는 114개 원소의 1차원 배열 입니다.
7~10	<code>accuracy_score(y_test, 각 pred)</code>	개별 모델의 성능을 측정합니다. 나중에 스태킹 결과와 비교 하기 위한 기준선입니다.

▶ 실행 결과

```
KNN 정확도: 0.9211
랜덤 포레스트 정확도: 0.9649
결정 트리 정확도: 0.9123
에이다부스트 정확도: 0.9737 :
```

■ 결과 해석

에이다부스트(0.9737)가 가장 우수하고 **결정 트리(0.9123)**가 가장 낮습니다. 단일 트리보다 앙상블(RF, AdaBoost)이 확실히 우수함을 다시 확인할 수 있습니다. 성능 차이가 6%p 정도 나는 **서로 다른 모델들**이라 스택킹의 재료로 적절합니다.

2.3 예측 결과를 피쳐로 변환

```
pred = np.array([knn_pred, rf_pred, dt_pred, ada_pred])
print(pred.shape)
```

```
# transpose를 이용해 행과 열의 위치 교환. 컬럼 레벨로 각 알고리즘의 예측 결과를 피쳐로 만듦.
pred = np.transpose(pred)
print(pred.shape)
```

줄	코드	상세 설명
1	pred = np.array([knn_pred, rf_pred, dt_pred, ada_pred])	4개 예측 배열을 하나의 2차원 배열로 쌓습니다. 결과 shape는 (4, 114) — 행이 모델, 열이 샘플입니다.
5	pred = np.transpose(pred)	이 실습의 핵심 단계입니다. 전치(transpose)로 행과 열을 바꿔 (114, 4)로 만듭니다. 이제 행이 샘플, 열이 모델이 되어, 머신러닝의 표준 데이터 형태(행=샘플, 열=피쳐)가 됩니다.
6	print(pred.shape)	변환 결과를 확인합니다.

▶ 실행 결과

```
(4, 114)
(114, 4)
```

■ 전치 후 데이터의 의미

샘플	피쳐1 (KNN 예측)	피쳐2 (RF 예측)	피쳐3 (DT 예측)	피쳐4 (Ada 예측)
0번 환자	1	1	0	1
1번 환자	0	0	0	0
...	...	...	...	...

원래 30개였던 피쳐가 4개로 바뀌었습니다. 각 피쳐는 "해당 모델이 이 환자를 어떻게 예측했는가"를 의미합니다. 메타 모델은 이 4개 값만 보고 최종 판단을 학습합니다.

2.4 메타 모델 학습과 평가

```
lr_final.fit(pred, y_test)
final = lr_final.predict(pred)

print('최종 메타 모델의 예측 정확도: {0:.4f}'.format(accuracy_score(y_test, final)))
```

줄	코드	상세 설명
1	lr_final.fit(pred, y_test)	메타 모델을 학습합니다. 입력은 4개 예측값, 정답은 y_test입니다. 여기에 심각한 문제가 있습니다(아래 참고).
2	final = lr_final.predict(pred)	학습에 사용한 바로 그 데이터로 예측합니다.
4	accuracy_score(y_test, final)	최종 정확도를 출력합니다.

▶ 실행 결과

```
최종 메타 모델의 예측 정확도: 0.9737
```

2.5 기본 스택킹의 치명적 문제

⚠ 이 코드는 개념 설명용이며 실무에서 사용하면 안 됩니다

위 코드에는 두 가지 심각한 문제가 있습니다.

1. 테스트 레이블(`y_test`)로 학습했습니다. `lr_final.fit(pred, y_test)` 에서 정답을 모델에 알려 준 것입니다. 실전에서는 테스트 데이터의 정답을 알 수 없으므로 애초에 불가능한 코드입니다.
2. 학습 데이터로 다시 평가했습니다. 학습에 쓴 `pred` 로 그대로 예측했으므로, 2.4장에서 배운 전형적인 과적합 측정입니다.

따라서 결과로 나온 0.9737은 신뢰할 수 없는 수치입니다. 에이다부스트 단독 성능과 같게 나온 것도 우연에 가깝습니다.

이 문제를 해결하는 것이 다음 절의 CV 세트 기반 스택킹이며, 실무에서 사용하는 진짜 스택킹입니다.

3. CV 세트 기반의 스택킹

3.1 개선 원리

CV 세트 기반 스택킹은 교차 검증을 이용해 메타 모델의 학습용 데이터를 만듭니다. 핵심은 테스트 레이블을 전혀 사용하지 않고 메타 모델을 학습시키는 것입니다.

단계	내용
스텝 1	각 기반 모델별로 K 폴드 교차 검증을 수행하며, ① 검증 폴드에 대한 예측을 모아 → 메타 모델의 학습 데이터 생성 ② 원본 테스트 데이터에 대한 예측을 폴드마다 수행해 평균 → 메타 모델의 테스트 데이터 생성
스텝 2	모든 기반 모델의 결과를 옆으로 이어 붙여(concatenate) 최종 학습/테스트 데이터를 만든 뒤, 메타 모델을 원본 학습 레이블(<code>y_train</code>)로 학습하고 원본 테스트 레이블(<code>y_test</code>)로 평가

■ 왜 이 방식이 올바른가

메타 모델의 학습 데이터가 "학습 과정에서 보지 않은 검증 폴드에 대한 예측"으로 만들어집니다. 즉 기반 모델이 처음 보는 데이터에서 어떻게 예측하는지를 학습하게 되므로, 실제 예측 상황과 조건이 같습니다. 그리고 정답은 `y_train`만 사용하므로 테스트 정보가 전혀 새지 않습니다.

3.2 기반 데이터 생성 함수

```
from sklearn.model_selection import KFold
from sklearn.metrics import mean_absolute_error

# 개별 기반 모델에서 최종 메타 모델이 사용할 학습 및 테스트용 데이터를 생성하기 위한 함수.
def get_stacking_base_datasets(model, X_train_n, y_train_n, X_test_n, n_folds):
    # 지정된 n_folds값으로 KFold 생성.
    kf = KFold(n_splits=n_folds, shuffle=False)
    #추후에 메타 모델이 사용할 학습 데이터 반환을 위한 넘파이 배열 초기화
    train_fold_pred = np.zeros((X_train_n.shape[0], 1))
    test_pred = np.zeros((X_test_n.shape[0], n_folds))
    print(model.__class__.__name__, ' model 시작 ')

    for folder_counter, (train_index, valid_index) in enumerate(kf.split(X_train_n)):
        #입력된 학습 데이터에서 기반 모델이 학습/예측할 폴드 데이터 셋 추출
        print('\t 폴드 세트: ', folder_counter, ' 시작 ')
```

```

X_tr = X_train_n[train_index]
y_tr = y_train_n[train_index]
X_te = X_train_n[valid_index]

#폴드 세트 내부에서 다시 만들어진 학습 데이터로 기반 모델의 학습 수행.
model.fit(X_tr , y_tr)
#폴드 세트 내부에서 다시 만들어진 검증 데이터로 기반 모델 예측 후 데이터 저장.
train_fold_pred[valid_index, :] = model.predict(X_te).reshape(-1,1)
#입력된 원본 테스트 데이터를 폴드 세트내 학습된 기반 모델에서 예측 후 데이터 저장.
test_pred[:, folder_counter] = model.predict(X_test_n)

# 폴드 세트 내에서 원본 테스트 데이터를 예측한 데이터를 평균하여 테스트 데이터로 생성
test_pred_mean = np.mean(test_pred, axis=1).reshape(-1,1)

#train_fold_pred는 최종 메타 모델이 사용하는 학습 데이터, test_pred_mean은 테스트 데이터
return train_fold_pred , test_pred_mean

```

줄	코드	상세 설명
5	def get_stacking_base_datasets(model, X_train_n, y_train_n, X_test_n, n_folds):	이 실습의 핵심 함수 입니다. 하나의 기반 모델에 대해 메타 모델용 학습/테스트 데이터를 생성합니다. <code>model</code> 을 인자로 받으므로 어떤 알고리즘이든 재사용 할 수 있습니다.
7	kf = KFold(n_splits=n_folds, shuffle=False)	K 폴드 객체를 생성합니다. <code>shuffle=False</code> 로 순서를 섞지 않아 결과를 재현할 수 있게 합니다.
9	train_fold_pred = np.zeros((X_train_n.shape[0], 1))	메타 모델의 학습 데이터를 담을 빈 배열 입니다. shape는 (학습 샘플 수, 1) = (455, 1). 각 학습 샘플에 대해 정확히 하나의 예측값 이 채워집니다.
10	test_pred = np.zeros((X_test_n.shape[0], n_folds))	메타 모델의 테스트 데이터를 담을 배열 입니다. shape는 (테스트 샘플 수, 폴드 수) = (114, 7). 폴드마다 테스트 예측을 따로 저장했다가 나중에 평균 냅니다.
13	for folder_counter, (train_index, valid_index) in enumerate(kf.split(X_train_n)):	폴드를 순회합니다. <code>enumerate</code> 로 폴드 번호 를 함께 얻어 <code>test_pred</code> 의 열 인덱스로 씁니다.
16~18	X_tr = X_train_n[train_index] 등	학습 데이터를 폴드 학습용(X_tr) 과 검증용(X_te) 으로 나눕니다. ndarray이므로 팬시 인덱싱이 바로 됩니다.
21	model.fit(X_tr, y_tr)	폴드 학습 데이터로 기반 모델을 학습합니다. 매 폴드마다 새로 학습 됩니다.
23	train_fold_pred[valid_index, :] = model.predict(X_te).reshape(-1,1)	가장 중요한 줄입니다. 검증 폴드(학습에 쓰이지 않은 데이터)에 대한 예측을 원래 위치(valid_index)에 그대로 저장합니다. <code>reshape(-1,1)</code> 은 1차원 예측 결과를 (n, 1) 2차원으로 바꿔 배열 형태를 맞추기 위함입니다. 7개 폴드를 모두 돌면 455개 학습 샘플 전부가 "학습에 쓰이지 않은 상태에서의 예측값" 으로 채워집니다.
25	test_pred[:, folder_counter] = model.predict(X_test_n)	원본 테스트 데이터 전체 를 매 폴드마다 예측해 해당 폴드 열에 저장합니다. 7개 폴드면 7번 예측 하게 됩니다.
28	test_pred_mean = np.mean(test_pred, axis=1).reshape(-1,1)	7개 폴드의 테스트 예측을 가로 방향(axis=1)으로 평균 냅니다. 여러 모델의 의견을 평균하는 효과로 더 안정적인 값 이 됩니다. 결과는 (114, 1)입니다.
31	return train_fold_pred, test_pred_mean	메타 모델용 학습 데이터와 테스트 데이터 를 함께 반환합니다.

⚠ 평균값이 소수가 되는 점에 주의

`test_pred_mean` 은 0/1 예측을 평균한 값이라 **0.43, 0.71 같은 소수**가 나올 수 있습니다. 예를 들어 7개 폴드 중 5개가 1로 예측하면 $5/7 = 0.714$ 가 됩니다. 이는 **일종의 확신도** 역할을 하므로 메타 모델에 유용한 정보가 됩니다. 다만 **학습 데이터(0/1 정수)와 분포가 달라진다는** 미묘한 불일치가 있어, 실무에서는 `predict_proba()` 를 사용해 **양쪽 모두 확률**로 통일하는 방식을 더 선호합니다.

3.3 4개 모델에 함수 적용

```
knn_train, knn_test = get_stacking_base_datasets(knn_clf, X_train, y_train, X_test, 7)
rf_train, rf_test = get_stacking_base_datasets(rf_clf, X_train, y_train, X_test, 7)
dt_train, dt_test = get_stacking_base_datasets(dt_clf, X_train, y_train, X_test, 7)
ada_train, ada_test = get_stacking_base_datasets(ada_clf, X_train, y_train, X_test, 7)
```

줄	코드	상세 설명
1~4	<code>get_stacking_base_datasets(각 모델, X_train, y_train, X_test, 7)</code>	4개 기반 모델 각각에 동일한 함수를 적용 합니다. 7폴드 로 설정했으므로 모델당 7번, 총 28번의 학습 이 수행됩니다. 각 호출은 (455,1) 학습 데이터와 (114,1) 테스트 데이터 를 반환합니다.

▶ 실행 결과

```
KNeighborsClassifier model 시작
폴드 세트: 0 시작
폴드 세트: 1 시작
폴드 세트: 2 시작
폴드 세트: 3 시작
폴드 세트: 4 시작
폴드 세트: 5 시작
폴드 세트: 6 시작
RandomForestClassifier model 시작
폴드 세트: 0 시작
... (7개 폴드) ...
DecisionTreeClassifier model 시작
... (7개 폴드) ...
AdaBoostClassifier model 시작
... (7개 폴드) ...
```

3.4 메타 모델용 데이터 결합

```
Stack_final_X_train = np.concatenate((knn_train, rf_train, dt_train, ada_train), axis=1)
Stack_final_X_test = np.concatenate((knn_test, rf_test, dt_test, ada_test), axis=1)
print('원본 학습 피쳐 데이터 Shape:', X_train.shape, '원본 테스트 피쳐 Shape:', X_test.shape)
print('스태킹 학습 피쳐 데이터 Shape:', Stack_final_X_train.shape,
      '스태킹 테스트 피쳐 데이터 Shape:', Stack_final_X_test.shape)
```

줄	코드	상세 설명
1	<code>np.concatenate((knn_train, rf_train, dt_train, ada_train), axis=1)</code>	4개의 (455,1) 배열을 옆으로 이어 붙여 (455, 4)로 만듭니다. <code>axis=1</code> 이 컬럼 방향 결합 을 의미합니다. 앞 절의 <code>transpose</code> 대신 이 방식을 쓰는 이유는 각 배열이 이미 (n, 1) 2차원 이기 때문입니다.
2	<code>Stack_final_X_test = np.concatenate(...)</code>	테스트 데이터도 동일하게 결합해 (114, 4)를 만듭니다. 컬럼 순서가 학습 데이터와 같아야 하므로 순서를 반드시 맞춥니다.
3~5	<code>print(...shape...)</code>	원본과 스택킹 데이터의 shape를 비교 출력합니다.

▶ 실행 결과

```
원본 학습 피쳐 데이터 Shape: (455, 30) 원본 테스트 피쳐 Shape: (114, 30)
스택킹 학습 피쳐 데이터 Shape: (455, 4) 스택킹 테스트 피쳐 데이터 Shape: (114, 4)
```

■ 결과 해석 - 차원 축소 효과

30개였던 피쳐가 4개로 압축되었습니다. 샘플 수(455, 114)는 그대로입니다. 4개의 기반 모델이 30개 피쳐의 정보를 각자 요약해 하나의 숫자로 만들어 낸 셈입니다. 메타 모델은 이 압축된 정보만으로 최종 판단을 학습합니다.

3.5 최종 메타 모델 학습과 평가

```
lr_final.fit(Stack_final_X_train, y_train)
stack_final = lr_final.predict(Stack_final_X_test)

print('최종 메타 모델의 예측 정확도: {0:.4f}'.format(accuracy_score(y_test, stack_final)))
```

줄	코드	상세 설명
1	lr_final.fit(Stack_final_X_train, y_train)	결정적인 차이입니다. 앞의 기본 스택킹은 <code>y_test</code> 로 학습했지만, 여기서는 <code>y_train</code> (원본 학습 레이블)로 학습합니다. 테스트 정보가 전혀 사용되지 않습니다.
2	stack_final = lr_final.predict(Stack_final_X_test)	학습에 쓰이지 않은 테스트 데이터로 예측합니다.
4	accuracy_score(y_test, stack_final)	신뢰할 수 있는 최종 정확도입니다.

▶ 실행 결과

```
최종 메타 모델의 예측 정확도: 0.9649
```

■ 결과 해석 - 전체 성능 비교

모델	정확도	비고
KNN	0.9211	개별 모델
결정 트리	0.9123	개별 모델 (최저)
랜덤 포레스트	0.9649	개별 모델
에이다부스트	0.9737	개별 모델 (최고)
기본 스택킹	0.9737	신뢰 불가 (y_test로 학습)
CV 세트 기반 스택킹	0.9649	올바른 방식의 결과

CV 세트 기반 스택킹은 **0.9649**로, 랜덤 포레스트와 같고 에이다부스트(0.9737)보다는 낮습니다.

스택킹이 항상 개별 모델보다 우수한 것은 아닙니다. 이유는 다음과 같습니다.

- 데이터가 작습니다(455건). 스택킹은 메타 모델 학습에 충분한 데이터가 필요합니다.
- 결정 트리(0.9123)처럼 성능이 낮은 모델이 포함되어 메타 모델에 노이즈를 더했습니다.
- 테스트가 114건이라 1건 차이가 0.88%p로 크게 반영됩니다. 이 정도 차이는 통계적으로 유의하지 않습니다.

실무에서는 수만 건 이상의 데이터와 성능이 비슷하면서 다양한 기반 모델들을 사용할 때 스택킹의 효과가 나타납니다. 그래서 캐글 등 0.1%의 성능을 다투는 대회에서 주로 쓰입니다.

4. 핵심 요약 및 머신러닝 활용 포인트

4.1 두 가지 스택킹 방식 비교

항목	기본 스택킹	CV 세트 기반 스택킹
메타 학습 데이터	테스트 예측 결과	검증 폴드 예측 결과
메타 학습 레이블	y_test (문제!)	y_train (올바름)
테스트 데이터	동일한 예측 결과 재사용	폴드별 예측의 평균
과적합 위험	매우 높음	낮음
실무 사용	불가 (개념 설명용)	가능

4.2 핵심 코드 패턴 요약

코드	역할	주의 사항
<code>np.array([pred1, pred2, ...])</code>	예측 결과 쌓기	결과는 (모델수, 샘플수)
<code>np.transpose(pred)</code>	행/열 교환	(샘플수, 모델수)로 만들어야 학습 가능
<code>np.concatenate((a,b,c,d), axis=1)</code>	컬럼 방향 결합	학습·테스트의 컬럼 순서 일치 필수
<code>np.zeros((n, 1))</code>	결과 저장 배열 초기화	shape를 미리 정확히 계산
<code>pred[valid_index, :] = ...</code>	검증 폴드 위치에 예측 저장	원래 인덱스 위치에 넣어야 정렬 유지

<code>np.mean(test_pred, axis=1)</code>	폴드별 예측 평균	<code>axis=1</code> 은 가로 방향
<code>.reshape(-1, 1)</code>	1차원 → (n,1) 2차원	배열 결합을 위해 필요

4.3 머신러닝 활용 포인트

- **기본 모델은 다양하게, 성능은 비슷하게** : 성격이 다르면서도 **성능 수준은 비슷한** 모델들을 모아야 효과가 좋습니다. 지나치게 약한 모델은 오히려 방해가 됩니다.
- **메타 모델은 단순하게** : 입력 피처가 몇 개뿐이므로 **로지스틱 회귀 같은 가벼운 선형 모델**이 적합합니다. 복잡한 모델을 쓰면 오히려 과적합됩니다.
- **데이터 누수를 항상 경계** : 스택킹은 구조가 복잡해 **테스트 정보가 새어 들어가기 쉽습니다**. 기본 스택킹 예제가 그 대표적인 사례입니다.
- **predict_proba 사용 고려** : 0/1 레이블 대신 **확률**을 메타 피처로 쓰면 정보량이 많아져 성능이 더 좋아지는 경우가 많습니다.
- **비용 대비 효과 판단** : 스택킹은 학습 횟수가 **모델 수 × 폴드 수**로 늘어납니다. 본 실습만 해도 28번을 학습했습니다. **얻는 성능 향상이 그만큼 가치가 있는지** 판단하세요.
- **사이킷런 내장 클래스 활용** : 0.22부터 `StackingClassifier` 가 제공되어 위 함수를 **직접 구현할 필요가 없습니다**(아래 참고).

⚠ 안정화 버전 참고 (scikit-learn 1.7.x 기준)

- 사이킷런 **0.22부터** `StackingClassifier` (회귀는 `StackingRegressor`)가 내장되어, 본 장의 복잡한 함수를 직접 구현할 필요가 없습니다. 교차 검증 처리와 데이터 누수 방지가 **자동으로** 이뤄집니다.

```
from sklearn.ensemble import StackingClassifier

estimators = [('knn', knn_clf), ('rf', rf_clf), ('dt', dt_clf), ('ada', ada_clf)]
stack_clf = StackingClassifier(estimators=estimators,
                              final_estimator=LogisticRegression(C=10), cv=7)
stack_clf.fit(X_train, y_train)
print(accuracy_score(y_test, stack_clf.predict(X_test)))
```

`stack_method='predict_proba'` 옵션으로 확률 기반 스택킹도 간단히 설정할 수 있습니다.

- `AdaBoostClassifier` 는 1.2부터 `base_estimator` 가 `estimator` 로 변경되었습니다. 또한 1.4부터 `algorithm='SAMME.R'` 이 deprecated되어, 1.6 이후로는 **'SAMME' 만** 사용됩니다.
- `DecisionTreeClassifier()` 에 `random_state` 를 지정하지 않으면 **실행할 때마다 결과가 조금씩 달라집니다**. 재현성이 필요하다면 반드시 지정하세요.